1. Importing Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import LabelEncoder
```

- **pandas (pd)**: For loading and manipulating datasets (CSV files).
- **train_test_split**: Splits the dataset into training and testing sets.
- **SVC**: The Support Vector Classifier from scikit-learn. This is the SVM algorithm.
- **confusion_matrix**: To evaluate model performance.
- **LabelEncoder**: Converts categorical labels (text) into numbers (not actually used in your current code).

---

2. Load Dataset

```
data = pd.read_csv('veriseti.csv', sep=",")
```

- Loads a CSV file named `veriseti.csv`.
- `sep=","` tells pandas that columns are separated by commas.
- After this, `data` is a **DataFrame**, like a table of rows and columns.

Example:

Feature1	Feature2	Target
65	5	1
72	6	0

---

3. Split into Training and Testing Sets

```
training_set, test_set = train_test_split(data, test_size=0.2,
random_state=1)
```

- Splits `data` into:
  - **training_set** → 80% of data
  - **test_set** → 20% of data
- `random_state=1` ensures reproducibility. Same split every time you run it.

---

4. Prepare Input (X) and Output (y)

```
x_train = training_set.iloc[:,0:2].values  # data
y_train = training_set.iloc[:,2].values    # target
```

```
x_test = test_set.iloc[:,0:2].values        # data
y_test = test_set.iloc[:,2].values          # target
```

- **`iloc[:,0:2]`** → selects the first two columns as features (`Feature1` and `Feature2`).
- **`iloc[:,2]`** → selects the third column as the target (label to predict).
- `.values` converts DataFrame slices into **numpy arrays**, which scikit-learn needs.

So your data is now ready for the SVM model.

---

5. Train the SVM Model

```
classifier = SVC(kernel='linear', random_state=1, C=1)
classifier.fit(x_train, y_train)
```

- `SVC` → creates an SVM classifier.
- **kernel='linear'** → uses a linear hyperplane for classification.
- **C=1** → regularization parameter (controls tradeoff between margin size and misclassification).
- `.fit()` → trains the SVM on `x_train` and `y_train`.

Now your model **learns the boundary** between the classes.

---

6. Make Predictions on Test Data

```
y_pred = classifier.predict(x_test)
```

- Uses the trained SVM to predict the target for `x_test`.
- `y_pred` is an array of predicted labels.

---

7. Evaluate Model Performance

```
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy = float(cm.diagonal().sum()) / len(y_test)
print('model accuracy is:', accuracy*100, '%')
```

- `confusion_matrix(y_test, y_pred)` → compares actual vs predicted labels.
    - Example output for binary classification:

```
[[8 2]
 [1 9]]
```

- Rows = actual class, columns = predicted class
- `cm.diagonal().sum()` → sum of correctly predicted samples

- Dividing by total number of test samples → gives **accuracy**.

---

8. Test with a New Data Point

```
x1_test = [[73,6]]  # new data
```

- Creates a new sample with features `73` and `6`.
- You can predict its class with:

```
new_prediction = classifier.predict(x1_test)
print(new_prediction)
```

This will output the predicted class (e.g., 0 or 1) for that new sample.